PES UNIVERSITY - DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SOFTWARE ENGINEERING (SE) LAB 4: VIBE CODING REPORT

Student Name:	Nagashri R Patil
SRN (USN):	PES1UG24CS289
Section & Semester:	Section E | 5th Semester B.Tech CSE
Assigned Repo:	https://github.com/SETAPESU26/43_reaction-time-tester (Project #43)
Date of Submission:	October 10, 2026

1. Executive Summary & Lab Objectives

Software Engineering Lab 4 (VibeCoding) emphasizes the practice of prompt-driven engineering and pairing with AI assistants to rapidly diagnose, isolate, and rectify software bugs, adding complex features in under 3-4 prompt attempts. The assigned repository is **43_reaction-time-tester** (Pygame-based reflex tester).

Deliverable	Location	Status
Videos (Before & After)	videos/video_before.mp4 videos/video_after.mp4	**COMPLETED** (10s Before, 12s After)
Updated Codebase	main.py, game/game_engine.py, game/round.py, game/sounds.py	**COMPLETED & TESTED** (All 4 Tasks)
Chat History Documentation	chat_history/ (PDF, DOCX, Markdown)	**COMPLETED** (Full Logs Attached)

2. Vibe Coding Prompt History & Iterative Engineering

Task 1: Refine Input Timing Detection & False Start Handling

Prompt:
```
In reaction-time-tester, reaction_ms is measured from self.start_time rather than self.go_time, inflating times by the wait delay. Also, clicking during the grey waiting phase is accepted as a valid reaction. Please refactor game/round.py and game/game_engine.py to: (1) measure genuine reactions strictly from self.go_time, and (2) detect premature clicks during waiting, set state to 'false_start', and display 'TOO EARLY!' without skewing the average.
```

LLM Solution: Refactored Round.update() to record go_time on state='go'. In register_input(), added branch: if waiting -> set false_start, play buzzer, return None. If go -> compute max(1, now - self.go_time). In GameEngine, rendered RED warning screen and ensured average excludes false starts.

Result: VERIFIED: Premature clicks trigger red warning screen and are excluded from averages; reactions after green cue measure accurate reflex speeds (~220ms).

Task 2: Implement Game Over Condition & Results Screen

Prompt:
```
Currently when rounds complete, game_engine.py only prints to console and freezes. Please build a graphical Game Over / Results screen in Pygame displaying: (1) Session Complete title, (2) Stats card with Average time, Best time, and False starts, (3) Round breakdown list, (4) Reflex performance rating (Elite/Pro/Good/Practice).
```

LLM Solution: Added _render_game_over() method in GameEngine with dark theme styling, stats card, best/average calculation, reflex rating classification, and round history breakdown pills.

Result: VERIFIED: Game gracefully transitions to results screen once round count is met, displaying complete breakdown and rating.

Task 3: Add Replay Option with Difficulty Selection

Prompt:
```
After results screen, allow user to replay with difficulty settings: Easy (2-4s wait, 3 rounds), Medium (1-3s wait, 5 rounds), Hard (0.5-2s wait, 7 rounds), or Exit. Support both keyboard keys (1, 2, 3, Q) and clickable on-screen buttons with hover glow.
```

LLM Solution: Added DIFFICULTIES config dictionary. Built _get_replay_buttons() and hover detection in handle_input(). In _handle_game_over_event(), bound number keys and mouse clicks to trigger reset_game(difficulty). Connected graceful exit via engine.should_quit.

Result: VERIFIED: Buttons glow on hover; clicking button or pressing key resets round counters and launches new session with chosen parameters immediately.

Task 4: Add Sound Feedback System

Prompt:
```
Add audio sound feedback for: (1) Go cue when screen turns green, (2) False start buzzer, (3) Game over fanfare. Ensure system is self-contained with no missing file crashes and supports headless fallbacks.
```

LLM Solution: Built game/sounds.py containing SoundManager with built-in procedural synthesis of 16-bit PCM WAV assets (go.wav 880Hz chime, false_start.wav 160Hz buzzer, game_over.wav 4-note fanfare). Hooked into Pygame mixer with robust exception handling.

Result: VERIFIED: Crisp sound playback on cues; safe fallback on non-audio systems.

3. Video Verification Deliverables

Video Artifact	Duration	Content & Behaviors Shown
videos/video_before.mp4	10s (300 frames)	Demonstrates buggy behavior: early click counted as reaction, inflated timing, no results screen.
videos/video_after.mp4	12s (360 frames)	Demonstrates fixed features: false start red screen, true reaction timing, results UI, and replay difficulty selection.

4. Execution Instructions

To run the game: navigate to **D:\sem5\se\se_project_n\VibeCoding\Lab4_N** and execute python main.py. Click or press SPACE when the screen turns green. On the Game Over screen, click difficulty buttons or press keys 1, 2, 3 to replay, or Q to exit.